

결측치 이해와 확인

빈 칸을 찾아내는 일이 분석의 첫 단추입니다

`isna` `na_values` `sum(axis=1)` 결측률

교안 14_03 마무리 · 15_01 결측치 이해와 확인 · 15_02 도입 | 53차시 | 2026. 08. 20. (목)

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
14_03 상관관계와 통합 리포트	상관 행렬 복습 · 실습 1~4 · 발견·해석·행동	완주
15_01 결측치 이해와 확인	전 범위 · 실습 1~5	완주
15_01 실습 6·7	결측 시각화 · 요약표 저장	미실시
15_02 결측치 제거와 대체	제거 vs 대체 · 행 삭제 vs 열 삭제 판단	개념만

이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

1. 분류 뱃지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 뱃지가 붙습니다. 이름만 외우면 나중에 반드시 헛갈립니다.

뱃지	무엇	판별 신호	오늘의 예
메서드	객체에 붙는 함수	점 뒤에 있고 괄호가 붙습니다	<code>.isna()</code> <code>.dropna()</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>.shape</code> <code>.columns</code>
함수	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>pd.read_csv()</code> <code>pd.DataFrame()</code>
키워드	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>na_values=</code> <code>axis=</code>
자료형	값의 종류	타입 이름입니다	<code>NaN</code> <code>None</code> <code>bool</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	결측률 · 위장 결측 · 전처리
패턴	자주 쓰는 코드 흐름	여러 요소의 조합입니다	<code>isna().sum()</code>

개념

3초 판별법. ① 앞에 점(.)이 있습니까? 없으면 함수나 키워드입니다. ② 점이 있다면 뒤에 괄호가 붙습니까? 붙으면 메서드, 안 붙으면 속성입니다. 오늘 가장 많이 쓸 `df.isna().sum()`을 이 기준으로 읽으면 `isna`도 메서드, `sum`도 메서드입니다. 반면 `df.shape`는 괄호가 없으니 속성입니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 출력은 상상해서 쓴 것이 아니라 실제 실행 결과를 옮긴 것입니다.

예시

```
import pandas as pd
```

```
df = pd.read_csv("Data/15_01_사출성형_공정.csv", encoding="utf-8")  
print(df.isna().sum().sum())
```

▶ 475

본문에서 이렇게 회색으로 칠해진 글자는 코드 조각입니다. **굵은 글씨**는 꼭 붙잡아야 할 문장입니다.

3. 이전 회차와의 연결

오늘은 **한 단원이 끝나고 새 단원이 시작되는 날**입니다. 앞부분에서 어제 배운 상관관계를 실습으로 마무리하며 지금까지 배운 집계 도구를 하나의 리포트로 묶었고, 뒷부분에서 완전히 새로운 주제인 **결측치**로 넘어갔습니다.

14 단원 마무리

집계 → 리포트



15_01

결측 확인



15_02

제거와 대체

중요한 점은 **지금까지 배운 것이 전부 "깨끗한 데이터"를 전제로 했다는 사실**입니다. 평균을 내고 상관계수를 구하는 모든 계산은 값이 다 채워져 있을 때 이야기입니다. 오늘부터는 **값이 비어 있는 현실의 데이터**를 다룹니다. 그래서 이 단원의 이름이 전처리입니다.

연결

어제 배운 `groupby().agg()`가 오늘 14_03 실습 4에서 **통합 리포트의 뼈대**로 쓰입니다. 그리고 15_01에서는 `isna()`가 만든 참·거짓 표에 `sum()`을 붙이는데, 이 `sum()`은 어제 배운 그 `sum()`과 같은 것입니다. **새 단원이라고 해서 도구가 새로 생기는 것은 아닙니다.** 이미 아는 도구를 새 대상에 붙이는 것입니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	한 단원의 마무리와 새 단원의 시작 — 두 주제의 연결점
STEP 1	상관 실습 마무리	실습 1·2 — 상관계수 읽기와 상관 행렬 다루기
STEP 2	그룹별 상관과 심슨의 역설	실습 3 — 전체와 그룹의 결론이 뒤집히는 경우
STEP 3	통합 리포트 — 발견·해석·행동	실습 4 — 숫자를 정비 판단으로 잇는 프레임
STEP 4	결측치란 무엇인가	NaN의 정체, 왜 생기나, 무엇이 문제인가
STEP 5	위장된 결측치	0 · -999 · 빈 문자열, 그리고 na_values
STEP 6	isna로 결측 세기	참·거짓 표와 sum, 컬럼별 개수와 비율
STEP 7	행 방향으로 보기	axis=1, 완전한 행, 결측 패턴 읽기
STEP 8	확인에서 판단으로	비율 기준, 그리고 제거 vs 대체
부록 A	치트시트	오늘 나온 모든 도구를 한 표에
부록 B	오류·함정 사전	제출 코드 교정 · 교안 예상 결과 대조
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 15_02 본론 예고

개념

오늘의 한 문장. **"빈 칸을 발견하지 못하면, 그 뒤의 모든 계산이 조용히 틀린다."** 결측이 있는 데이터로 평균을 내면 에러가 나지 않습니다. 그냥 **빠진 값을 뻔 채로 계산된 숫자**가 나옵니다. 오류 메시지가 없으니 눈치채기 어렵고, 그래서 더 위험합니다.

오늘의 지도

한 단원을 닫고 새 단원을 여는 날

0-1. 오늘 수업의 두 덩어리

강사님은 수업을 "어제 배웠던 내용, 키워드 어떤 것들이 생각나나요?"라는 질문으로 시작하셨습니다. agg를 짚고 넘어간 뒤 14_03의 남은 실습을 마무리했고, 그 다음 곧바로 15단원 결측치로 넘어갔습니다.

시간대	교안	내용	이 책
앞부분	14_03	상관 행렬 복습 · 실습 1~4 · 통합 리포트	STEP 1~3
중반	15_01	결측치 개념 · 위장 결측 · isna · 비율	STEP 4~6
후반	15_01	행 방향 확인 · 실습 5 · 처리 판단의 기초	STEP 7~8
끝부분	15_02	제거 vs 대체 · 행 삭제 vs 열 삭제 판단	STEP 8-3

0-2. 왜 전처리를 배우는가

개념 데이터 전처리 (preprocessing) 분석 전에 데이터를 다듬는 모든 작업

정의 모델을 만들기 전에 데이터를 깨끗하게 다듬는 과정입니다. 결측치 처리, 이상치 처리, 정규화 등이 모두 여기 들어갑니다.

왜 쓰나 교안이 제시한 수치가 인상적입니다. 분석 업무 시간의 70~80%가 전처리에 쓰입니다. 멋진 모델을 만드는 시간보다 데이터를 손질하는 시간이 훨씬 깁니다.

응용 그 이유는 현장 데이터가 깨끗한 적이 없기 때문입니다. 센서가 고장 나고, 통신이 끊기고, 사람이 입력을 빠뜨립니다. 오늘 실습에서 쓸 사출성형 공정 데이터를 보면 이 사정이 바로 드러납니다.

```
import pandas as pd
```

```
df = pd.read_csv("Data/15_01_사출성형_공정.csv", encoding="utf-8")
```

```
print("전체 칸 수:", df.shape[0] * df.shape[1])
```

```
print("빈 칸 수 :", df.isna().sum().sum())
```

```
print("결측 비율 :", round(df.isna().sum().sum() / (df.shape[0] * df.shape[1]) * 100, 1), "%")
```

▶ 전체 칸 수 : 5500

▶ 빈 칸 수 : 475

▶ 결측 비율 : 8.6 %

개념

교안의 비유가 정확합니다. "쓰레기를 넣으면 쓰레기가 나온다(Garbage In, Garbage Out)." 재료 손질이 부실하면 아무리 좋은 요리법을 써도 요리가 망합니다. 전처리하는 요리 실력이 아니라 **재료 손질**입니다.

설비 적용

설비 데이터는 특히 결측이 흔합니다. 센서가 물리적 부품이라 언제든 고장 나고, 점검·교체 중에는 기록이 끊기고, 먼지와 고온 때문에 일시적으로 측정에 실패하기도 합니다. **결측은 사고가 아니라 일상**입니다.

0-3. 오늘 다루는 데이터

파일	행 × 열	성격	어디에 쓰나
14_hydraulic_qc.csv	200 × 11	품질검사 지표	14_03 실습 1~3
14_equipment_sensor.csv	120 × 7	라인·교대별 센서	14_03 실습 4
15_사출성형_로그.csv	16 × 7	작은 로그 — 눈으로 볼 수 있는 크기	15_01 실습 1·3
15_01_사출성형_공정.csv	250 × 22	본격 공정 데이터 — 결측 475개	15_01 실습 2·4·5

설비 적용

15_사출성형_로그.csv는 **16행짜리 장난감 데이터**입니다. 크기가 작아 눈으로 전부 확인할 수 있으니 개념을 익히기에 좋습니다. 반면 15_01_사출성형_공정.csv는 **250행 22열에 결측이 475개**나 되어 눈으로는 절대 파악할 수 없습니다. 교안의 표현대로 "**장난감 데이터 졸업**"입니다. 오늘 배울 도구들은 바로 이 두 번째 상황을 위한 것입니다.

사출성형은 플라스틱 원료를 녹여 금형에 밀어 넣고 굳혀 제품을 만드는 공정입니다. 배럴온도로 원료를 녹이고, 스크루가 밀어 넣고, 사출압력으로 금형을 채웁니다. 한 사이클이 약 16.6초이고, 250번의 사이클이 이 데이터에 담겨 있습니다.

0-4. 한 사이클에서 무엇을 재는가

22개 컬럼의 이름만 봐서는 무엇이 무엇인지 알기 어렵습니다. 사출성형의 한 사이클을 따라가면서 각 항목이 어느 단계에서 나오는지 정리하면 **결측 패턴을 읽는 데 결정적인 도움**이 됩니다.

단계	하는 일	이 데이터의 컬럼
① 가열	배럴에서 원료를 녹입니다	배럴온도1~4 · 호퍼온도
② 사출	스크루가 녹은 원료를 금형에 밀어 넣습니다	최대사출속도 · 사출압력 · 최대사출압
③ 전환	속도 제어에서 압력 제어로 바뀝니다	전환위치 · 전환압력 · 스크루위치
④ 보압·냉각	압력을 유지하며 굳힙니다	최소쿠션 · 감압시간
⑤ 계량	다음 사이클용 원료를 스크루 앞에 모읍니다	계량시간 · 계량시작점 · 계량시작위치 · 계량종료점

이 표를 손에 쥐고 6-3의 결측 순위를 다시 보면 흥미로운 사실이 드러납니다. **결측이 거의 없는 컬럼은 앞 단계(가열·사출)에 몰려 있고, 결측이 많은 컬럼은 뒤 단계(보압·계량)에 몰려 있습니다.** 계량종료점과 감압시간의 결측률이 43.6%로 가장 높은 것도 이 둘이 **사이클의 맨 끝에서 측정되는 항목**이기 때문입니다.

설비 적용

사이클이 중간에 끊기면 **그 이후 단계의 측정값이 통째로 기록되지 않습니다.** 그래서 뒤로 갈수록 결측이 늘어나는 계단 모양이 나옵니다. 이런 구조를 알고 있으면 **"뒤쪽 센서가 부실하다"가 아니라 "사이클이 자주 중단된다"**로 해석이 바뀝니다. 같은 숫자를 보고도 전혀 다른 조치를 취하게 되는 것입니다.

STEP 1

상관 실습 마무리

실습 1 · 2 — 상관계수 읽기와 상관 행렬 다루기

어제 배운 `corr()`을 실습으로 굳히는 시간이었습니다. 강사님은 "어제 배웠던 내용, 키워드 어떤 것들이 생각나나요?"로 시작해 `agg`를 다시 짚고, 곧바로 상관 실습으로 들어가셨습니다.

1-1. 실습 1 — 두 열의 상관계수 구하기

Practice/14_03_example.py — 실습 1

```
import pandas as pd
import os

path_1 = os.path.join("Data", "14_hydraulic_qc.csv")
df_1 = pd.read_csv(path_1, encoding="utf-8")

# corr로 두 지표의 상관계수 구하기
r_1 = df_1["지표07"].corr(df_1["지표08"])
print("corr():", r_1)

# round로 소수점 셋째 자리까지 정리해 읽기 쉽게 만들기
print("corr().round(3):", r_1.round(3))

▶ corr(): -0.9690877323579701
▶ corr().round(3): -0.969
```

반올림하지 않은 원래 값은 소수점 아래가 열여섯 자리까지 이어집니다. 이 숫자를 그대로 보고서에 쓰는 사람은 없습니다. `round(3)`은 계산을 바꾸는 것이 아니라 읽기 쉽게 만드는 것입니다.

개념

제출 코드의 주석이 해석을 정확하게 적었습니다. "-0.969 → 부호가 음수이므로 지표07이 오르면 지표08은 내려감 → 절댓값이 1에 매우 가까우므로 강한 음의 상관관계." 어제 배운 "부호는 방향, 절댓값은 강도"를 그대로 적용한 문장입니다. 상관계수를 읽을 때는 항상 이 두 조각으로 나눠 읽으십시오.

반올림을 붙이는 두 가지 방법

강사님은 시연에서 같은 결과를 내는 네 가지 표기를 나란히 보여주셨습니다. 어느 것을 써도 되지만 읽기 좋은 위치가 따로 있습니다.

Python/29_correlation_report.py — 강사 시연

```
cor1 = df["온도"].corr(df["진동"])

print(cor1)                                # 그대로
print(cor1.round(3))                       # 변수에 붙이기
print(round(cor1, 3))                      # 내장 round 쓰기
print(round(df["온도"].corr(df["진동"]), 3)) # 한 줄로
print(df["온도"].corr(df["진동"]).round(3)) # 체이닝으로
```

```
▶ 0.9307966383117151
▶ 0.931
▶ 0.931
▶ 0.931
▶ 0.931
```

주의

`.round(3)`이 붙을 수 있는 이유는 `corr()`의 결과가 **넘파이 실수(float64)**이기 때문입니다. 파이썬 기본 실수에는 `.round()` 메서드가 없습니다. 순수 파이썬 값이라면 `round(값, 3)` 형태의 **내장 함수**를 써야 합니다. 판다스·넘파이 안에서만 메서드 형태가 통한다는 점을 기억해 두십시오.

1-2. 실습 2 — 상관 행렬로 여러 열 비교

열이 열 개면 조합이 45가지입니다. 한 줄씩 쓰면 45줄이 되니 `corr()`을 **데이터프레임 전체에** 붙여 표로 받습니다.

Practice/14_03_example.py — 실습 2

```
cols = ["지표01", "지표02", "지표03", "지표04", "지표05",
        "지표06", "지표07", "지표08", "지표09", "지표10"]

corr_matrix = df_2[cols].corr()
print(corr_matrix.round(2))
```

```
▶ 지표01  지표02  지표03  지표04  지표05  지표06  지표07  지표08  지표09  지표10
▶ 지표01  1.00  0.94  0.72 -0.98 -0.90  0.33 -0.88  0.76 -0.96  1.00
▶ 지표02  0.94  1.00  0.80 -0.94 -0.93  0.48 -0.92  0.84 -0.96  0.94
▶ 지표03  0.72  0.80  1.00 -0.71 -0.95  0.89 -0.96  0.99 -0.88  0.70
▶ 지표04 -0.98 -0.94 -0.71  1.00  0.89 -0.33  0.87 -0.75  0.95 -0.98
▶ 지표05 -0.90 -0.93 -0.95  0.89  1.00 -0.71  1.00 -0.97  0.98 -0.88
▶ ... (지표06~10 생략)
```

표를 읽는 규칙은 어제 배운 그대로입니다. 대각선은 항상 1.00이니 건너뛰고, 대각선 기준으로 위아래가 대칭이니 한쪽 삼각형만 봅니다. 그리고 절댓값이 큰 칸에 집중합니다.

주의

round(2)로 줄였더니 지표01-지표10이 1.00으로 표시됩니다. 완벽하게 같다는 뜻일까요. 자릿수를 늘려 보면 실제 값은 0.999입니다. 두 지표가 사실상 같은 것을 재고 있다는 뜻이지 완전히 동일한 것은 아닙니다. 반올림은 읽기를 돕지만 동시에 정보를 지웁니다. 1.00이 보이면 반드시 자릿수를 늘려 확인하십시오.

1-3. 한 열만 뽑아 정렬하기

패턴 corr_matrix["열"].sort_values() 특정 지표와 가장 강하게 묶인 상대 찾기

정의 상관 행렬에서 한 열만 꺼내면 그 지표와 나머지 모든 지표의 상관계수가 담긴 Series가 됩니다. 여기에 정렬을 붙이면 순위표가 나옵니다.

형태 df[cols].corr()["기준지표"].sort_values(ascending=False)

```
# 특정 열 하나만 뽑아 다른 지표들과의 상관을 정렬해서 확인
print(corr_matrix["지표01"].sort_values(ascending=False).round(2))
```

▶ 지표01	1.00	← 자기 자신
▶ 지표10	1.00	
▶ 지표02	0.94	
▶ 지표08	0.76	
▶ 지표03	0.72	
▶ 지표06	0.33	
▶ 지표07	-0.88	
▶ 지표05	-0.90	
▶ 지표09	-0.96	
▶ 지표04	-0.98	
▶ Name: 지표01, dtype: float64		

왜 쓰나 45칸짜리 표를 눈으로 훑는 대신 관심 있는 지표 하나를 기준으로 줄 세워 봅니다. "지표01과 가장 밀접한 것은 무엇인가"라는 질문에 한 줄로 답합니다.

응용 정렬 결과의 양 끝이 모두 중요합니다. 맨 위 지표10(+1.00)은 가장 강한 양의 관계, 맨 아래 지표04(-0.98)는 가장 강한 음의 관계입니다. 가운데 있는 지표06(0.33)이 오히려 관계가 가장 약합니다. 강도로 줄 세우려면 절댓값으로 정렬해야 합니다.

절댓값 기준으로 다시 정렬 — 강도 순

```
s = corr_matrix["지표01"].drop("지표01")  
print(s.reindex(s.abs().sort_values(ascending=False).index).round(2))
```

▶ 지표10	1.00	
▶ 지표04	-0.98	
▶ 지표09	-0.96	
▶ 지표02	0.94	
▶ 지표05	-0.90	
▶ 지표07	-0.88	
▶ 지표08	0.76	
▶ 지표03	0.72	
▶ 지표06	0.33	← 가장 약함

개념

`drop("지표01")`로 자기 자신을 먼저 뺐습니다. 자기 자신과의 상관 1.00은 언제나 맨 위에 오므로 **순위표에서는 의미 없는 잡음**입니다.

주의

교안 p20은 이중 반복문으로 모든 쌍을 훑는 방법을 제시했지만, 제출 코드는 **한 열만 뽑아 정렬하는 방법**을 택했습니다. 둘 다 맞습니다. 반복문 방식은 전체 쌍을 빠짐없이 보고, 한 열 방식은 **관심 지표가 정해져 있을 때 훨씬 간단합니다**. 목적에 따라 고르십시오.

STEP 2

그룹별 상관과 심슨의 역설

실습 3 — 전체와 그룹의 결론이 뒤집히는 경우

실습 3은 어제 예고했던 문제입니다. 같은 두 지표의 상관이 그룹에 따라 달라지는지 확인하는 것인데, 결과가 상당히 극적입니다.

2-1. 전체 · 합격 · 불합격을 나눠 보기

Practice/14_03_example.py — 실습 3

```
# 전체 데이터의 지표07과 지표08 상관계수
r_all = df_3["지표07"].corr(df_3["지표08"])
print("전체 :", r_all.round(3))

# 판정 옰로 합격 그룹을 나눠 같은 두 지표의 상관계수 계산
df_pass = df_3[df_3["검사결과"] == "합격"]
r_pass = df_pass["지표07"].corr(df_pass["지표08"])
print("합격 :", r_pass.round(3), "| 표본 수:", len(df_pass))

# 판정 옰로 불합격 그룹을 나눠 같은 두 지표의 상관계수 계산
df_fail = df_3[df_3["검사결과"] == "불합격"]
r_fail = df_fail["지표07"].corr(df_fail["지표08"])
print("불합격 :", r_fail.round(3), "| 표본 수:", len(df_fail))
```

▶ 전체 : -0.969
▶ 합격 : 0.385 | 표본 수: 188
▶ 불합격 : -0.998 | 표본 수: 12

구분	상관계수	표본 수	읽는 법
전체	-0.969	200	강한 음의 관계
합격	+0.385	188	부호가 뒤집혔습니다
불합격	-0.998	12	거의 완벽한 반비례 — 다만 12건

전체를 보면 -0.969로 강한 음의 관계인데, **합격 188건만 떼어 보면 +0.385로 부호가 반대가 됩니다.** 전체의 94%를 차지하는 합격 그룹에서 관계가 정반대인데도 전체 결론이 음수인 것입니다.

2-2. 왜 이런 일이 생기는가

개념

이것이 통계에서 **심슨의 역설(Simpson 역설)**이라 부르는 현상입니다. 전체를 볼 때와 그룹으로 나눠 볼 때 결론이 정반대가 되는 경우를 가리킵니다. 원인은 대개 하나입니다. **그룹마다 값의 중심 위치가 크게 다르기** 때문입니다.

이 데이터에서 무슨 일이 벌어졌는지 직접 확인해 보겠습니다.

그룹별 중심과 흩어짐

```
print(df_3.groupby("검사결과")[["지표07", "지표08"]].mean().round(2))
print(df_3.groupby("검사결과")[["지표07", "지표08"]].std().round(2))
```

```
▶ 지표07    지표08
▶ 검사결과
▶ 불합격  36.50   118.48
▶ 합격    58.90   109.46
▶
▶ 지표07    지표08
▶ 검사결과
▶ 불합격  12.13    7.81
▶ 합격    0.32    0.26
```

답이 보입니다. **불합격 그룹은 지표07이 낮고(36.50) 지표08이 높습니다(118.48).** 합격 그룹은 정반대입니다(**58.90 / 109.46**). 두 그룹이 대각선 방향으로 멀찍이 떨어져 있으니, 둘을 한꺼번에 놓고 보면 **"07이 낮으면 08이 높다"**는 강한 음의 관계로 보입니다.

그런데 그것은 **그룹 사이의 차이**이지 **그룹 안의 관계**가 아닙니다. 합격 그룹 안에서 지표07의 표준편차는 0.32밖에 안 됩니다. 188개의 점이 거의 한자리에 뭉쳐 있으니 그 안에서의 관계는 약할 수밖에 없습니다. 반대로 불합격 그룹은 표준편차가 12.13으로 **합격의 38배**나 되어 넓게 퍼져 있습니다.

설비 적용

현장에서 이런 함정은 **여러 설비·여러 라인의 데이터를 한 표에 합쳤을 때** 자주 생깁니다. A 설비는 저온·저압으로, B 설비는 고온·고압으로 돌아간다면, 둘을 합쳐 온도-압력 상관을 구하면 **설비 차이가 만들어 낸 가짜 상관**이 나옵니다. 상관을 구하기 전에 **"이 데이터가 하나의 모집단인가"**를 먼저 물어야 합니다.

2-3. 표본 수를 반드시 함께 보기

불합격 그룹의 -0.998 은 거의 완벽한 반비례입니다. 그런데 표본이 12건뿐입니다. 12개의 점으로 낸 상관계수를 얼마나 믿어도 될까요.

직접 실험해 보면 답이 나옵니다. **상관이 0.385로 약한 합격 그룹**에서 12건씩 무작위로 뽑아 상관계수를 계산해 보는 것입니다.

12건으로 낸 상관계수는 얼마나 흔들리는가

```
import numpy as np

p = df_3[df_3["검사결과"] == "합격"]      # 전체 188건, 실제 상관 0.385
vals = []
for i in range(2000):
    s = p.sample(12, random_state=i)      # 12건씩 무작위로 뽑기
    vals.append(s["지표07"].corr(s["지표08"]))

v = np.array(vals)
print("범위   :", round(v.min(), 3), "~", round(v.max(), 3))
print("|r| >= 0.7   :", round((abs(v) >= 0.7).mean() * 100, 1), "%")
print("|r| >= 0.9   :", round((abs(v) >= 0.9).mean() * 100, 1), "%")
print("부호가 음수 :", round((v < 0).mean() * 100, 1), "%")
```

```
▶ 범위   : -0.909 ~ 0.982
▶ |r| >= 0.7 : 46.7 %
▶ |r| >= 0.9 : 6.7 %
▶ 부호가 음수 : 18.1 %
```

결과가 충격적입니다. 실제 상관이 0.385에 불과한 집단에서 12건만 뽑았더니, 절반 가까운 경우(46.7%)에 절댓값 0.7 이상의 "강한 상관"이 나왔습니다. 0.9를 넘는 경우도 6.7%였고, 부호가 아예 반대로 뒤집힌 경우도 18.1%였습니다.

주의

그러므로 불합격 12건의 -0.998 을 "두 지표가 거의 완벽하게 반대로 움직인다"고 읽으면 안 됩니다. 표본이 적으면 아무 관계가 없어도 강한 상관이 우연히 만들어집니다. 제출 코드의 주석이 이 점을 정확히 짚었습니다. "불합격은 12건뿐이라 표본이 적어 값이 과장돼 보일 수 있음 → 신중히 해석." 20일차의 "저하-고부하 3건의 평균은 신중히"와 같은 교훈입니다. **통계량 옆에는 항상 표본 수를 적으십시오.**

제출 코드가 `len(df_pass)`와 `len(df_fail)`을 함께 출력한 것은 좋은 습관입니다. 표본 수를 출력하지 않았다면 -0.998 만 보고 강한 결론을 내렸을 것입니다.

STEP 3

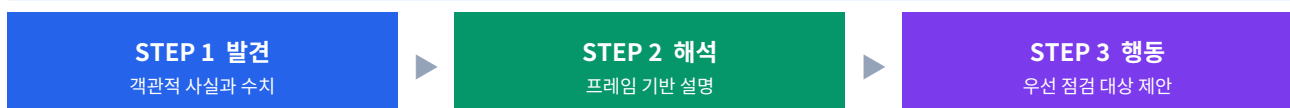
통합 리포트 — 발견 · 해석 · 행동

실습 4 — 숫자를 정비 판단으로 잇는 프레임

14단원의 마지막 실습입니다. 지금까지 배운 **빈도 · 그룹 통계 · 상관관계**를 하나의 리포트로 묶어 우선 점검 대상을 제안하는 것이 목표입니다.

강사님은 리포트 작성에 대해 한 가지 원칙을 강조하셨습니다. **"판다스를 돌려 봤다 하는 게 있으면 그걸 우선해서 위쪽에다가 배치해 버려야 돼요. 왜냐하면 그 서류를 보는 사람 입장에서 생각해 본다면..."** 즉 읽는 사람이 가장 먼저 알아야 할 것을 맨 위에 놓으라는 뜻입니다.

3-1. 리포트의 세 단계



단계	무엇을 쓰나	쓰면 안 되는 것
발견	집계로 확인한 객관적 사실과 구체적 수치	아직 해석하지 않기
해석	평균·표준편차·상관 강도 프레임으로 의미 설명	원인을 단정하지 않기
행동	우선 점검 대상과 모니터링 항목 제안	근거 없는 제안

3-2. 실습 4 — 라인별 진단표와 상관

Practice/14_03_example.py — 실습 4

```
path_4 = os.path.join("Data", "14_equipment_sensor.csv")
df_4 = pd.read_csv(path_4, encoding="utf-8")

# 라인으로 그룹을 나눠 측정수·평균온도·온도편차 요약
report_4 = (
    df_4.groupby("line")
        .agg(측정수=("temp", "count"), 평균온도=("temp", "mean"), 온도편차=("temp",
"std"))
        .round(2)
)
print(report_4.sort_values("온도편차", ascending=False))

# 온도와 진동의 상관계수를 구해 함께 움직이는지 확인
```

```
print("온도-진동 corr:", df_4["temp"].corr(df_4["vibration"]).round(3))
```

```
# 고장 행만 걸러 라인별 고장 건수 집계
```

```
df_bad = df_4[df_4["result"] == "고장"]  
print(df_bad.groupby("line").size())
```

```
▶ 측정수 평균온도 온도편차  
▶ line  
▶ C라인 31 79.88 10.38  
▶ A라인 54 76.86 10.18  
▶ B라인 35 77.69 7.60  
▶  
▶ 온도-진동 corr: 0.345  
▶  
▶ line  
▶ A라인 16  
▶ B라인 6  
▶ C라인 6  
▶ dtype: int64
```

제출 코드가 마지막에 붙여 둔 **최종 해석**이 이 실습의 답입니다. 그대로 옮기면 이렇습니다.

개념

발견 — 온도편차는 C라인(10.38)이 가장 크지만, 고장 건수는 A라인(16건)이 가장 많습니다.

해석 — 온도와 진동의 상관은 0.345로 약한 양의 관계이므로 온도만으로 고장을 설명하기 어렵습니다.

행동 — 편차가 큰 C라인과 고장이 잦은 A라인을 우선 점검 대상으로 분리해 확인합니다.

세 문장이 각각 제 역할을 하고 있습니다. 발견에는 숫자만 있고, 해석에는 "**설명하기 어렵다**"는 신중한 표현이 쓰였으며, 행동은 두 라인을 **분리해서** 보라는 구체적 제안입니다. 좋은 리포트입니다.

3-3. 여기에 한 줄만 더 보탠다면

발견 문장을 한 걸음 더 밀어붙일 수 있습니다. **고장 건수는 A라인이 16건으로 가장 많지만, 라인마다 측정 건수가 다릅니다.** 어제 배운 대로 건수가 아니라 비율로 봐야 합니다.

건수를 비율로 환산하기

```
t = df_4.groupby("line").agg(측정수=("result", "size"))
t["고장건수"] = df_4[df_4["result"] == "고장"].groupby("line").size()
t["고장률%"] = (t["고장건수"] / t["측정수"] * 100).round(1)
print(t)
```

```
▶ 측정수 고장건수 고장률%
▶ line
▶ A라인  54      16      29.6
▶ B라인  35       6      17.1
▶ C라인  31       6      19.4
```

다행히 이 데이터에서는 건수 순위와 비율 순위가 같습니다. A라인이 건수로도 16건으로 1위이고 비율로도 29.6%로 1위입니다. 그러니 원래 결론이 유지됩니다. 다만 B라인과 C라인은 고장 건수가 똑같이 6건인데 **비율로는 17.1%와 19.4%로 갑니다.** C라인의 측정 건수가 더 적기 때문입니다.

주의

건수 순위와 비율 순위가 우연히 같았을 뿐이라는 점이 중요합니다. 어제 20일차에서는 밸브상태별 고장 건수가 비슷해 보였지만 비율로는 네 배 차이가 났습니다. **어느 쪽이 나올지는 미리 알 수 없으므로 항상 둘 다 확인하는 편이 안전합니다.** 리포트에는 비율을 신고 괄호 안에 건수를 병기하는 방식이 실무에서 흔합니다.

한 가지 더 짚자면, **C라인의 온도편차 10.38과 A라인의 10.18은 사실상 같은 값**입니다. 0.2 차이는 전체 온도 표준편차에 비하면 무시할 수준입니다. 정렬해서 C라인이 맨 위에 왔을 뿐, **"C라인이 유독 불안정하다"**고 말하기는 어렵습니다. 실제로 뚜렷하게 다른 것은 B라인(7.60)이 나머지 둘보다 **안정적이라는 사실**입니다.

STEP 4

결측치란 무엇인가

NaN의 정체 · 왜 생기나 · 무엇이 문제인가

여기서부터 새 단원입니다. 지금까지의 모든 계산은 **값이 다 채워져 있다는 가정** 위에 있었습니다. 이제 그 가정을 걷어냅니다.

4-1. NaN의 정체

자료형 NaN 판다스가 빈 칸에 붙이는 값 없음 뜻말

정의 Not a Number의 줄임말입니다. 데이터 표에서 값이 비어 있는 칸을 발견하면 판다스가 자동으로 찍어 두는 표시입니다.

형태 `import numpy as np → np.nan`

```
import pandas as pd
```

```
df = pd.read_csv("Data/15_사출성형_로그.csv", encoding="utf-8")
print(df.head(4))
```

	측정시각	사출기	배럴온도	사출압력	스크루속도	누적샷	불량여부
▶ 0	2024-05-07 08:00	1호기	201.2	1.32	45.1	1200	0
▶ 1	2024-05-07 08:01	1호기	201.5	1.33	45.0	1201	0
▶ 2	2024-05-07 08:02	1호기	NaN	1.31	44.9	1202	0
▶ 3	2024-05-07 08:03	1호기	202.0	0.00	45.2	1203	1

왜 쓰나 CSV 파일을 메모장으로 열면 결측 자리는 그냥 빈 칸입니다. 그 파일을 판다스로 불러오는 순간 빈 칸이 NaN으로 바뀝니다. 도구마다 옷만 다를 뿐 속은 같습니다.

응용 2행의 배럴온도가 NaN입니다. 사람은 이 줄을 그냥 넘어가지만 컴퓨터는 넘어가지 못합니다. 평균을 내려 하면 계산이 멈추거나, 조용히 그 값을 빼고 계산해 버립니다.

```
# NaN이 섞인 열의 평균 — 오류가 나지 않는다는 것이 함정
print("배열온도 값 개수:", df["배열온도"].count())
print("전체 행 수 :", len(df))
print("배열온도 평균 :", df["배열온도"].mean().round(2))
```

▶ 배열온도 값 개수: 14
▶ 전체 행 수 : 16
▶ 배열온도 평균 : 257.9

개념

오류가 나지 않습니다. 16행짜리 데이터인데 14개만 가지고 평균을 냈습니다. 화면에는 아무 경고도 뜨지 않습니다. 이것이 결측이 위험한 진짜 이유입니다. **에러는 눈에 띄지만 조용한 오차는 눈에 띄지 않습니다.**

주의

평균이 257.9로 나온 것도 이상합니다. 다른 배열온도는 198~204도인데 말이지요. 범인은 8행의 **999.0**입니다. 이 한 값을 결측으로 바꾸면 평균이 **200.89로 내려갑니다.** 999 하나가 평균을 57도나 밀어 올린 것입니다. 이것이 다음 장에서 다룰 **위장 결측**입니다.

4-2. NaN과 None의 차이

구분		
출신	순수 파이썬	넘파이 · 판다스
의미	아무것도 없음	숫자가 아님 (빈 숫자 칸)
쓰이는 곳	일반 파이썬 코드	데이터 표의 숫자 컬럼
판다스에서	둘 다 똑같이 결측으로 잡힙니다	isna()가 모두 True로 표시

개념

교안의 답이 명쾌합니다. **"둘을 구분해야 하나요? 초보 단계에서는 구분할 필요가 없습니다."** isna()가 둘 다 잡아 주기 때문입니다. 이름이 두 개인 것은 **판다스가 파이썬과 넘파이 양쪽을 다 받아들이기 때문**이지, 처리 방법이 다르기 때문이 아닙니다.

4-3. 결측은 왜 생기는가

빈 칸을 발견했을 때 **"왜 빠졌을까"**를 먼저 묻는 습관이 중요합니다. 이유를 알면 채울지 지울지 판단이 쉽니다.

원인	상황	자연스러운 처리
센서 고장	센서가 망가져 그 순간부터 값이 빈	통째로 고장 난 컬럼은 제거 검토
점검 · 교체	센서를 떼는 동안 기록이 끊김	짧은 구간이면 앞뒤 값으로 채움
환경 요인	먼지 · 고온으로 일시적 측정 실패	대체로 채워 살림
통신 끊김	센서에서 서버로 오는 길에 사라짐	발생 시점 확인 후 판단
표 합치기	1분 단위와 5분 단위를 합쳐 시점이 안 맞음	합치는 규칙을 다시 검토

설비 적용

강사님이 수업 중 든 예가 인상적입니다. **"센서에서 자꾸 데이터가 오다가 한 번은 데이터가 안 들어오는 것 같아요. 절반. 크게 데이터가 빈 채로 오는데, 이거 센서 자체 문제가 있는 거 아닌가."** 결측률이 유난히 높은 컬럼을 발견하면 그것은 **데이터 문제가 아니라 설비 문제의 신호일** 수 있습니다. 데이터 분석이 설비 진단으로 이어지는 지점입니다.

4-4. 결측을 방치하면 벌어지는 일

영역	증상	왜
평균 왜곡	10개 중 2개가 결측이면 8개 평균이 됨	빠진 2개가 유난히 높았다면 실제보다 낮게 보임
전 통계 영향	합계 · 표준편차 · 상관계수도 함께 흔들림	모든 통계가 같은 값들로 계산되기 때문
그래프 끊김	선이 뚝 끊기거나 점이 듄성듬성	그릴 값이 없으니 자리를 비움
모델 거부	학습을 시작조차 못 하고 에러로 중단	대부분 머신러닝 도구가 빈 칸을 받지 않음

개념

교안의 비유가 정확합니다. **"잉크 한 방울이 깨끗한 물 전체를 흐린다."** NaN이 하나만 끼어도 그 값을 쓰는 **모든 계산 결과가 NaN이 되거나 빠진 채로 계산됩니다. 이것을 결측의 전염성이라고 부릅니다. 그래서 분석의 맨 앞에서 처리해야 합니다.**

4-5. 등호로 결측을 찾으려면 안 되는 이유

개념 NaN의 비교 연산 자기 자신과 비교해도 다르다고 나오는 별난 값

정의 NaN은 자기 자신과 비교해도 거짓을 돌려줍니다. 그래서 등호로는 결측을 찾을 수 없습니다.

```
import numpy as np

print(np.nan == np.nan)
print(np.nan != np.nan)

# 그래서 이 방법은 언제나 0을 준다
print((df["배럴온도"] == np.nan).sum())
```

- ▶ False ← 자기 자신과도 다르다고 나옴
- ▶ True
- ▶ 0 ← 분명 빈 칸이 있는데 0개로 보임

왜 쓰나 초보가 가장 자주 당하는 함정입니다. `df["온도"] == NaN`을 써 놓고 결과가 0이니 "결측이 없구나" 하고 넘어가면, 그 뒤의 모든 분석이 틀립니다.

응용 왜 이렇게 만들었을까요. "값이 없는 것"과 "값이 없는 것"이 같다고 단정하기가 어색하기 때문입니다. 온도 기록이 빠진 것과 압력 기록이 빠진 것이 같은 사건일 리 없습니다. 그래서 결측 전용 도구인 `isna()`가 따로 있습니다.

```
# 올바른 방법 — isna를 쓴다
print(df["배럴온도"].isna().sum())
```

- ▶ 2 ← 제대로 잡힘

자주 하는 실수

`== np.nan`뿐 아니라 `== None`도 마찬가지입니다. **결측은 반드시 `isna()`로 찾습니다. 예외는 없습니다.**

STEP 5

위장된 결측치

0 · -999 · 빈 문자열, 그리고 na_values

진짜 결측치는 `isna()`가 잡아 줍니다. 문제는 **멀쩡한 숫자인 척 숨어 있는 결측치**입니다. 이것을 찾아내는 것이 오늘 실습의 절반을 차지했습니다.

5-1. 위장 결측치의 세 가지 얼굴

유형	예	정체	판단
숫자 0	사출압력 = 0.0	센서가 죽어 0을 보낸 것	맥락으로 판단
센티넬 값	-999 999	옛 시스템이 정한 값 없음 표시	거의 확실히 결측
빈 문자열·공백	" " "	글자 컬럼에서 값이 빠진 것	문자열 정리 필요

개념

센티넬 값(sentinel value)이란 "값이 없음" 자리에 넣기로 **약속한 현실에 없는 숫자**입니다. 온도 -999도는 물리적으로 불가능하니 결측 표시로 쓰기 좋습니다. 옛날 시스템은 빈 칸을 저장할 방법이 없어 이런 약속을 만들었고, 그 관행이 지금도 남아 있습니다.

가장 까다로운 것은 **0**입니다. 교안이 지정한 대로 **똑같은 0이라도 어떤 건 결측이고 어떤 건 정상**입니다. 가동 중인 설비의 사출압력이 진짜 0일 수는 없으니 위장 결측이지만, 스크루 속도가 0이라면 잠시 멈춘 정상 상태일 수 있습니다. 컴퓨터는 이 구분을 못 합니다. 데이터를 아는 사람이 판단해야 합니다.

5-2. 실습 1 — 진짜 결측과 위장 결측 나눠 세기

Practice/15_01_example.py — 실습 1

```
path_1 = os.path.join("Data", "15_사출성형_로그.csv")
df_1 = pd.read_csv(path_1, encoding="utf-8")

# isna로 컬럼별 NaN 개수 세기
print(df_1.isna().sum())
print("전체 NaN 개수:", df_1.isna().sum().sum())

# 조건 필터링으로 위장 결측 개수 세기
print("사출압력 == 0.0      :", (df_1["사출압력"] == 0.0).sum())
print("스크루속도 == -999.0:", (df_1["스크루속도"] == -999.0).sum())
print("배럴온도 == 999.0   :", (df_1["배럴온도"] == 999.0).sum())
```

```
▶ 측정시각    0
▶ 사출기      1
▶ 배럴온도    2
▶ 사출압력    1
▶ 스크루속도  1
▶ 누적샷      0
▶ 불량여부    0
▶ dtype: int64
▶ 전체 NaN 개수: 5
▶
▶ 사출압력 == 0.0      : 2
▶ 스크루속도 == -999.0: 2
▶ 배럴온도 == 999.0   : 1
```

결과가 명확합니다. **진짜 결측 5개, 위장 결측 5개**. `isna()`만 믿었다면 절반을 놓쳤을 것입니다. 이 16행짜리 데이터에서는 **결측이 실제로는 열 개**입니다.

둘을 합쳐 보기

```
real_na = df_1.isna().sum().sum()
fake_na = (
    (df_1["사출압력"] == 0.0).sum()
    + (df_1["스크루속도"] == -999.0).sum()
    + (df_1["배럴온도"] == 999.0).sum()
)
print("진짜 결측(NaN):", real_na, "| 위장 결측:", fake_na, "| 합계:", real_na + fake_na)
```

```
▶ 진짜 결측(NaN): 5 | 위장 결측: 5 | 합계: 10
```

주의

위장 결측을 세는 조건식이 **컬럼마다 다르다**는 점에 주목하십시오. 사출압력은 `== 0.0`, 스크루속도는 `== -999.0`, 배럴온도는 `== 999.0`입니다. 어떤 값이 위장 결측인지는 컬럼마다 다르고, 데이터를 직접 들여다봐야 알 수 있습니다. 자동으로 알아내는 방법은 없습니다. `describe()`의 최솟값·최댓값이 그 단서가 됩니다.

5-3. na_values — 불러올 때부터 결측으로 바꾸기

키워드 `na_values=` 지정한 값을 읽는 순간 NaN으로 변환

정의 `read_csv()`에 넘기는 인자입니다. 목록에 적은 값들을 파일에서 읽어 들이는 순간 NaN으로 바꿔 줍니다.

형태 `pd.read_csv("파일", na_values=[-999, 999])`

```
# 변환 전 - 그냥 불러오기
df_before = pd.read_csv(path_3, encoding="utf-8")
print("배럴온도 == 999.0 :", (df_before["배럴온도"] == 999.0).sum())
print("스크루속도 == -999.0:", (df_before["스크루속도"] == -999.0).sum())
print("NaN 개수:", df_before.isna().sum().sum())

# na_values로 위장값을 결측으로 인식해 다시 불러오기
df_after = pd.read_csv(path_3, encoding="utf-8", na_values=[-999, 999])
print("배럴온도 == 999.0 :", (df_after["배럴온도"] == 999.0).sum())
print("스크루속도 == -999.0:", (df_after["스크루속도"] == -999.0).sum())
print("NaN 개수:", df_after.isna().sum().sum())
```

```
▶ 배럴온도 == 999.0 : 1
▶ 스크루속도 == -999.0: 2
▶ NaN 개수: 5
▶
▶ 배럴온도 == 999.0 : 0 ← 사라짐
▶ 스크루속도 == -999.0: 0 ← 사라짐
▶ NaN 개수: 8 ← 5에서 3 늘어남
```

왜 쓰나 다 불러온 뒤에 하나씩 바꾸는 것보다 **간편하고 실수가 적습니다**. 그리고 한 번 NaN이 되면 이후의 모든 결측 도구가 통째로 적용됩니다.

응용 변환 전후를 표로 비교하면 어느 컬럼에서 몇 개가 바뀌었는지 한눈에 보입니다. 제출 코드가 이 비교표를 직접 만들었습니다.


```
compare_3 = pd.DataFrame({
    "변환전": df_before.isna().sum(),
    "변환후": df_after.isna().sum(),
})
compare_3["증가"] = compare_3["변환후"] - compare_3["변환전"]
print(compare_3)
```

```
▶ 변환전 변환후 증가
▶ 측정시각  0    0    0
▶ 사출기    1    1    0
▶ 배럴온도  2    3    1  ← 999.0 하나
▶ 사출압력  1    1    0
▶ 스크루속도 1    3    2  ← -999.0 둘
▶ 누적샷   0    0    0
▶ 불량여부  0    0    0
```

개념

`pd.DataFrame({...})`으로 두 **Series**를 나란히 붙여 표를 만드는 방법입니다. 딕셔너리의 키가 열 이름이 되고, 인덱스(컬럼 이름)가 자동으로 맞춰집니다. 비교표를 만들 때 아주 자주 쓰는 패턴입니다.

주의

`na_values=[-999, 999]`는 **모든 컬럼에** 적용됩니다. 만약 어떤 컬럼에서 999가 진짜 정상값이라면 그것까지 지워 버립니다. 컬럼마다 다르게 지정하려면 딕셔너리를 넘깁니다. `na_values={"배럴온도": [999], "스크루속도": [-999]}` 형태입니다.

5-4. na_values로도 안 잡히는 것

제출 코드가 마지막에 아주 중요한 확인을 했습니다.

남은 위장 결측

```
# na_values로 다 잡히지 않는 위장 결측도 있음
print("변환 후에도 남은 사출압력 == 0.0 :", (df_after["사출압력"] == 0.0).sum())
```

```
▶ 변환 후에도 남은 사출압력 == 0.0 : 2
```

`na_values`에 0을 넣지 않았기 때문입니다. **일부러 넣지 않은 것입니다.** 제출 코드의 주석이 그 이유를 정확히 적었습니다. **"0이 진짜 측정값일 수도 있어서 도메인 판단이 필요한 부분."**

자주 하는 실수

`na_values=[0, -999, 999]`처럼 **0까지 한꺼번에 넣어 버리는 것**입니다. 그러면 누적샷 0, 불량여부 0 같은 **정상값들이 전부 결측으로 바뀝니다**. 이 데이터에서 불량여부는 0이 정상 제품을 뜻하는데, 0을 결측 처리하면 정상 제품 기록이 통째로 사라집니다. `na_values`는 그 값이 절대 정상일 수 없는 경우에만 씁니다.

5-5. 위장 결측을 찾는 실전 순서

위장 결측은 자동으로 찾을 수 없다고 했습니다. 그렇다고 눈으로만 찾을 수도 없습니다. **단서를 얻는 순서**가 있습니다.

순서	도구	무엇을 보나	의심 신호
1	<code>describe()</code>	각 열의 min과 max	-999 · 999 · 0이 끝값에 있음
2	<code>value_counts()</code>	유난히 자주 나오는 값	한 값이 비정상적으로 많음
3	<code>head()</code> <code>tail()</code>	실제 값의 모양	같은 열인데 자릿수가 튼
4	공정 지식	물리적으로 가능한 값인가	가동 중 압력 0 · 온도 999도

끝값만 뽑아 보기

```
# 1단계 — describe의 min·max로 훑기
print(df_1[["배럴온도", "사출압력", "스크루속도"].describe().loc[["min", "max"]])
```

```
▶   배럴온도 사출압력 스크루속도
▶ min   198.10   0.00  -999.00   ← 압력 0, 속도 -999
▶ max   999.00   1.42   47.20   ← 온도 999
```

`describe()` 전체를 출력하면 여덟 줄이 나오지만, `.loc[["min", "max"]]`로 **두 줄만 뽑으면** 위장 결측 단서가 한눈에 들어옵니다. 세 개가 전부 걸렸습니다. 배럴온도 최댓값 999, 사출압력 최솟값 0, 스크루속도 최솟값 -999입니다.

주의

4단계인 **공정 지식**이 결국 결정적입니다. 배럴온도 999도는 플라스틱을 녹이는 온도가 아니라 강철이 녹는 온도이니 명백한 위장 결측입니다. 반면 사출압력 0은 **판단이 필요합니다**. 설비가 가동 중이었다면 결측이지만, 그 사이클에 실제로 사출이 안 이뤄졌다면 진짜 0입니다. **코드가 답을 주지 않는 지점이 반드시 남습니다**.

STEP 6

isna로 결측 세기

참·거짓 표와 sum · 컬럼별 개수와 비율

여기서부터는 **250행 22열짜리 본격 데이터**를 씁니다. 눈으로는 절대 파악할 수 없는 크기입니다.

6-1. isna의 동작 원리

메서드 `.isna()` 결측이면 True, 값이 있으면 False

정의 `is na` — 결측이니?를 묻는 전용 도구입니다. 원본과 **같은 크기**의 참·거짓 표를 만들어 돌려줍니다.

형태 `df.isna()` | `df["열"].isna()`

```
df = pd.read_csv("Data/15_사출성형_로그.csv", encoding="utf-8")
print(df[["배럴온도", "사출압력"]].isna().head(4))
```

```
▶ 배럴온도 사출압력
▶ 0  False  False
▶ 1  False  False
▶ 2   True   False   ← 배럴온도가 빈 칸
▶ 3  False  False
```

왜 쓰나 4-5에서 본 대로 등호로는 결측을 찾을 수 없습니다. `isna()`는 NaN과 None을 모두 정확히 True로 잡아 줍니다.

등호의 함정을 해결하는 유일한 방법입니다.

응용 결과가 원본과 **같은 크기**라는 점이 핵심입니다. 250행 22열이면 참·거짓도 250행 22열입니다. 그래서 이 표에 어떤 방향으로 `sum()`을 붙이느냐에 따라 컬럼별 결측 개수도, 행별 결측 개수도 얻을 수 있습니다.

```
big = pd.read_csv("Data/15_01_사출성형_공정.csv", encoding="utf-8")
print("원본 크기   :", big.shape)
print("isna 결과 크기 :", big.isna().shape)
print("자료형      :", big.isna().dtypes.unique())
```

```
▶ 원본 크기   : (250, 22)
▶ isna 결과 크기 : (250, 22)
▶ 자료형      : [dtype('bool')]
```

개념

`isna()`의 결과는 **모든 칸이 bool 자료형**입니다. 원본이 숫자든 글자든 상관없이 전부 참·거짓으로 바뀝니다. 그래서 다음에 배울 `sum()`이 통합니다.

주의

`isnull()`은 `isna()`와 **완전히 같은 기능**입니다. 판다스 역사적 이유로 같은 기능에 이름이 두 개일 뿐입니다. 반대로 값이 있으면 참인 것은 `notna()`(또는 `notnull()`)입니다. **초보**는 `isna()` 하나만 쓰면 충분합니다.

6-2. True는 1 — sum으로 세기

패턴 `isna().sum()` 결측 확인의 핵심 공식

정의 참·거짓 표에 `sum()`을 붙이면 **True의 개수**가 나옵니다. 컴퓨터는 아주 오래전부터 **참을 1, 거짓을 0**으로 다뤄왔기 때문입니다.

형태 `df.isna().sum()`

```
# True + False + True + True = 1 + 0 + 1 + 1 = 3
print(big.isna().sum())
```

```
▶ 측정시각      0
▶ 불량여부      0
▶ 사이클시간    0
▶ ... (결측 없는 열 생략)
▶ 사출압력      1
▶ 스크루위치     3
▶ 전환위치      5
▶ 계량시간      9
▶ 계량시작점     9
▶ 최대사출압    34
▶ 전환압력      34
▶ 계량시작위치   34
▶ 스크루속도    60
▶ 최소쿠션      68
▶ 계량종료점    109
▶ 감압시간      109
▶ dtype: int64
```

왜 쓰나 한 줄로 데이터 전체의 결측 지도를 그립니다. 22개 컬럼을 하나씩 확인하는 대신 왼쪽에 컬럼 이름, 오른쪽에 결측 개수가 세로로 나열됩니다.

응용 `sum()`을 한 번 더 붙이면 전체 결측 개수가 나옵니다. 컬럼별 개수를 다시 더하는 것입니다.

```
print("전체 결측 개수:", big.isna().sum().sum())
print("전체 칸 수 :", big.shape[0] * big.shape[1])
print("전체 결측률 :", round(big.isna().sum().sum() / (big.shape[0] * big.shape[1])
* 100, 1), "%")
```

▶ 전체 결측 개수: 475
▶ 전체 칸 수 : 5500
▶ 전체 결측률 : 8.6 %

개념

`isna().sum()`은 **한 단어처럼 외우십시오**. 데이터를 받으면 `info()` 다음으로 반드시 실행하는 명령입니다. 결측이 어디에 얼마나 있는지 모르는 채로 분석을 시작하면 나중에 반드시 되돌아오게 됩니다.

설비 적용

왜 컬럼별로 볼까요. 컬럼은 **하나의 센서 하나의 항목**이기 때문입니다. 특정 컬럼에 결측이 몰려 있다면 그 센서 자체가 의심 대상이고, 여러 컬럼에 골고루 퍼져 있다면 통신이나 저장 과정의 문제일 가능성이 큼니다.

6-3. 실습 2 — `info`와 `describe`로 첫인상 잡기

`isna()`를 쓰기 전에도 결측의 존재를 눈치챌 방법이 있습니다. 데이터를 처음 받았을 때 가장 먼저 실행하는 두 가지입니다.

```
df_2 = pd.read_csv("Data/15_01_사출성형_공정.csv", encoding="utf-8")
```

```
print(df_2.shape)
print(df_2.info())
```

```

> (250, 22)
>
> RangeIndex: 250 entries, 0 to 249
> Data columns (total 22 columns):
> #   Column      Non-Null Count  Dtype
> ---  ---
> 0   측정시각    250 non-null    object
> 9   최대사출속도 250 non-null    float64
> 10  사출압력    249 non-null    float64  ← 250보다 작다
> 18  스크루속도  190 non-null    float64
> 20  계량종료점  141 non-null    float64
> 21  감압시간    141 non-null    float64

```

개념

`info()`의 **Non-Null Count**가 결측 탐지의 첫 단서입니다. 전체가 250행인데 어떤 컬럼이 249로 표시되면 **1개가 빈 칸**이라는 뜻입니다. 강사님도 이 지점을 짚으셨습니다. "**250으로 다 채워진 게 0번 컬럼부터 9번 컬럼까지는 다 나오는데, 그 이후에 있는 거 보니까 숫자가 250이 좀 안 되죠. 이거는 무슨 의미겠어요? NaN으로 찍혀 있거나 한 결측 데이터가 그만큼 포함이 돼 있다는 뜻이겠죠.**"

`describe()`도 같은 일을 합니다. 맨 윗줄의 **count**가 **결측이 아닌 값의 개수**이기 때문입니다. 게다가 `describe()`는 **최솟값과 최댓값도 보여주기 위장 결측의 단서**까지 함께 잡을 수 있습니다. `min`이 -999이거나 `max`가 999라면 즉시 의심해야 합니다.

결측 있는 컬럼만

```
# 결측이 있는 컬럼만 추려서 내림차순 확인
na_count_2 = df_2.isna().sum()
print(na_count_2[na_count_2 > 0].sort_values(ascending=False))
```

```
▶ 감압시간    109
▶ 계량종료점   109
▶ 최소쿠션     68
▶ 스크루속도   60
▶ 전환압력     34
▶ 계량시작위치  34
▶ 최대사출압   34
▶ 계량시간      9
▶ 계량시작점    9
▶ 전환위치      5
▶ 스크루위치     3
▶ 사출압력      1
▶ dtype: int64
```

na_count_2[na_count_2 > 0]은 어제 배운 **불리언 인덱싱**입니다. Series에도 똑같이 통합니다. 22개 컬럼 중 **결측이 있는 것은 12개로** 좁혀졌습니다.

설비 적용

제출 코드의 주석이 아주 좋은 관찰을 담았습니다. "**뒤쪽 센서 컬럼일수록 채워진 수가 줄어드는 패턴이 보임.**"

실제로 결측 개수가 1 → 3 → 5 → 9 → 34 → 60 → 68 → 109로 **계단처럼 늘어납니다**. 이런 패턴은 우연이 아니라 **데이터가 만들어진 방식**을 알려 줍니다. 뒤쪽 항목일수록 측정이 늦게 이뤄지고, 사이클이 중간에 끊기면 뒤쪽부터 기록되지 않았을 가능성이 큼니다.

6-4. 실습 4 — 개수가 아니라 비율로

같은 isna() 표에 무엇을 붙이느냐로 답이 달라집니다. 셋을 나란히 놓고 보면 관계가 분명해집니다.

붙이는 것	묻는 질문	결과 예 (감압시간)	언제 쓰나
.sum()	몇 개가 비었나	109	절대량 파악
.mean()	몇 %가 비었나	0.436	처리 방향 판단
.count()	몇 개가 살아 있나	141	신뢰도 확인

패턴 `isna().mean()` 컬럼별 결측 비율을 한 줄로

정의 `isna()`가 참·거짓이라 **평균을 내면 곧 True의 비율**이 됩니다. 즉 결측률입니다.

형태 `df.isna().mean()` | `df.isna().mean() * 100`

```
# isna().mean()으로 컬럼별 결측 비율 구하기
na_rate_4 = df_4.isna().mean()
print(na_rate_4[na_rate_4 > 0].round(3))
```

```
▶ 사출압력    0.004
▶ 스크루위치  0.012
▶ 전환위치    0.020
▶ 계량시간    0.036
▶ 계량시작점  0.036
▶ 최대사출압  0.136
▶ 전환압력    0.136
▶ 계량시작위치 0.136
▶ 스크루속도  0.240
▶ 최소쿠션    0.272
▶ 계량종료점  0.436
▶ 감압시간    0.436
▶ dtype: float64
```

왜 쓰나 개수만 보면 처리 방향이 안 보입니다. 결측 109개가 250행짜리 데이터에서는 43.6%로 심각하지만, 40,000행짜리 데이터에서는 0.3%로 무시할 수준입니다. **같은 숫자가 전혀 다른 의미가 됩니다.**

응용 `sum()`과 `mean()`은 **같은 참·거짓 표에 붙는 형제**입니다. 무엇을 물을지에 따라 골라 쓰면 됩니다. 그리고 100을 곱하면 퍼센트가 되고, 개수와 나란히 놓으면 처리 지시서가 됩니다. 제출 코드가 만든 표가 그것입니다.


```
report_4 = pd.DataFrame({
    "결측수": df_4.isna().sum(),
    "결측률(%)": (df_4.isna().mean() * 100).round(1),
})
report_4 = report_4[report_4["결측수"] > 0].sort_values("결측률(%)",
ascending=False)
print(report_4)
```

```
▶   결측수 결측률(%)
▶ 감압시간  109    43.6
▶ 계량종료점 109    43.6
▶ 최소쿠션   68    27.2
▶ 스크루속도  60    24.0
▶ 전환압력   34    13.6
▶ 계량시작위치 34    13.6
▶ 최대사출압  34    13.6
▶ 계량시간    9     3.6
▶ 계량시작점   9     3.6
▶ 전환위치     5     2.0
▶ 스크루위치   3     1.2
▶ 사출압력     1     0.4
```

개념

`isna().mean()`이 비율이 되는 이유를 다시 짚겠습니다. True가 1, False가 0이니 **평균 = (1의 개수) / (전체 개수) = 비율**입니다. `sum()`을 쓰면 개수, `mean()`을 쓰면 비율. **메서드 하나만 바꾸면 됩니다.**

주의

열 이름에 (%) 같은 괄호가 들어가면 `report_4.결측률` 형태의 점 접근이 **불가능합니다**. 반드시 `report_4["결측률(%)"]`처럼 대괄호를 써야 합니다. 이 표를 만든 코드가 계속 대괄호를 쓴 이유입니다.

6-5. 기준선을 정해 우선순위 나누기

비율을 구했으면 **기준선을 그어 처리 방향을 나눕니다**. 이것이 확인 단계의 마지막이자 판단 단계의 시작입니다.

결측 비율	처리 방향	이유	이 데이터의 해당 컬럼
5% 미만	대체로 살리기	빠진 게 적으니 메워도 신뢰도가 안 흔들림	사출압력·스크루위치·전환위치·계량시간·계량시작점
5 ~ 30%	중요도 보고 결정	분석에 꼭 필요한 컬럼인지 따져 봄	최대사출압·전환압력·계량시작위치·

결측 비율	처리 방향	이유	이 데이터의 해당 컬럼
			스크루속도 · 최소쿠션
40% 이상	제거 고민	채워 봤자 절반이 추측 — 오히려 왜곡	계량종료점 · 감압시간

사용 여부 재검토 대상

기준선을 정해 처리 우선순위 나누기

```
print(report_4[report_4["결측률(%)"] >= 40])
```

▶ 결측수 결측률 (%)

▶ 감압시간 109 43.6

▶ 계량종료점 109 43.6

주의

비율은 출발점이지 결론이 아닙니다. 결측률 45%인 컬럼이라도 그것이 분석에 반드시 필요한 핵심 지표라면

함부로 버릴 수 없습니다. 반대로 결측률 3%여도 애초에 쓸모없는 컬럼이면 채울 이유가 없습니다. **비율 · 중요도 ·**

결측 원인 · 정답 컬럼 여부를 함께 따져야 합니다.

STEP 7

행 방향으로 보기

axis=1 · 완전한 행 · 결측 패턴 읽기

지금까지는 **세로**로 봤습니다. "어느 센서가 많이 비었나." 이제 시각을 90도 돌립니다. "어느 측정 기록이 많이 비었나."

7-1. axis로 방향 바꾸기

키워드 axis=1 가로 방향으로 계산하라는 지시

정의 sum()이나 mean() 같은 집계 메서드에 넘기는 인자입니다. axis=0(기본값)은 세로, axis=1은 가로 방향입니다.

형태 df.isna().sum(axis=1)

```
# axis=1로 방향을 바꿔 행마다 결측이 몇 개인지 세기
na_per_row = df_5.isna().sum(axis=1)
print(na_per_row.head(10))
```

```
▶ 0    5
▶ 1    2
▶ 2    0   ← 완전한 행
▶ 3    0
▶ 4    4
▶ 5    1
▶ 6    4
▶ 7    1
▶ 8    1
▶ 9    1
▶ dtype: int64
```

왜 쓰나 컬럼별 확인은 어느 센서를 버릴까를 묻고, 행별 확인은 어느 측정 기록을 버릴까를 묻습니다. 결측 처리에서 이 두 방향의 판단이 모두 필요합니다.

응용 행마다 결측 개수를 알면 **완전한 행이 몇 개인지** 셀 수 있습니다. 이 숫자가 곧 "결측 있는 행을 다 지우면 몇 줄이 남는가"의 답입니다.

```
print("전체 행 수 :", len(df_5))
print("결측 있는 행 수:", df_5.isna().any(axis=1).sum())
print("완전한 행 수 :", (~df_5.isna().any(axis=1)).sum())
print("완전한 행 비율:", round((~df_5.isna().any(axis=1)).mean() * 100, 1), "%")
```

- ▶ 전체 행 수 : 250
- ▶ 결측 있는 행 수: 174
- ▶ 완전한 행 수 : 76
- ▶ 완전한 행 비율: 30.4 %

개념

`any(axis=1)`은 그 행에 **True**가 하나라도 있으면 **True**를 돌려줍니다. 즉 "결측이 하나라도 있는 행"입니다. 앞에 `~`를 붙이면 뒤집혀 "결측이 전혀 없는 행"이 됩니다. `~`는 지난 시간에 배운 **부정 연산자**입니다.

주의

`axis` 인자는 헛갈리기로 유명합니다. 외우는 요령은 "**axis는 없애는 축**"입니다. `sum(axis=1)`은 열 방향(가로)을 없애 각 행에 숫자 하나만 남깁니다. 그래서 결과가 250개짜리 Series가 됩니다.

7-2. 완전한 행이 30.4%뿐이라는 뜻

개념

250행 중 결측이 전혀 없는 행은 76행뿐입니다. 나머지 174행에는 어딘가 빈 칸이 있습니다. 이 숫자가 왜 중요할까요. "결측이 있는 행을 다 지우자"고 결정하면 데이터의 70%가 사라진다는 뜻이기 때문입니다. 확인 단계에서 이 숫자를 미리 알아야 그런 성급한 결정을 피할 수 있습니다.

행별 결측 개수의 분포를 보면 상황이 더 선명합니다.

결측이 몰려 있나 퍼져 있나

```
# 행별 결측 개수의 분포 확인
print(na_per_row.value_counts().sort_index())
```

```
▶ 0      76   ← 완전한 행
▶ 1      45
▶ 2      41
▶ 3      49
▶ 4      12
▶ 5      13
▶ 6      11
▶ 7       2
▶ 8       1   ← 가장 부실한 행
▶ Name: count, dtype: int64
```

결측이 **특정 행에 몰려 있지 않고 골고루 퍼져 있습니다**. 결측 1~3개인 행이 135개로 대부분이고, 8개나 비어 있는 행은 단 하나입니다. 어제 배운 `value_counts()`를 여기서 다시 씁니다. **분포를 볼 때는 언제나 이 도구입니다**.

설비 적용

만약 결측이 **특정 시간대에 몰려 있었다면** 이야기가 완전히 달라집니다. 정전이나 설비 정지 같은 **공통 사건**을 의심해야 하고, 그 구간을 통째로 빼는 편이 나을 수 있습니다. 반대로 지금처럼 골고루 퍼져 있다면 **개별 센서의 산발적 실패**로 보는 편이 자연스럽고, 행을 지우기보다 값을 채우는 쪽이 유리합니다.

7-3. 결측 패턴 — 같이 비는 컬럼들

6-3에서 결측 개수가 계단처럼 늘어난다고 했습니다. 그런데 자세히 보면 **똑같은 숫자가 반복됩니다**. 109가 두 개, 34가 세 개, 9가 두 개입니다. 우연일까요.

동시 결측 그룹 찾기

```
# 결측 위치가 완전히 같은 열끼리 묶어 보기
na = df_5.isna()
groups = {}
for c in df_5.columns:
    if na[c].sum() == 0:
        continue

    key = tuple(na[c].values.nonzero()[0]) # 결측인 행 번호 목록
    groups.setdefault(key, []).append(c)

for key, cols in sorted(groups.items(), key=lambda x: -len(x[0])):
    print(f"{len(key):3d}건 : {cols}")
```

- ▶ 109건 : ['계량종료점', '감압시간']
- ▶ 68건 : ['최소쿠션']
- ▶ 60건 : ['스크루속도']
- ▶ 34건 : ['최대사출압', '전환압력', '계량시작위치']
- ▶ 9건 : ['계량시간', '계량시작점']
- ▶ 5건 : ['전환위치']
- ▶ 3건 : ['스크루위치']
- ▶ 1건 : ['사출압력']

우연이 아니었습니다. **계량종료점과 감압시간은 109개 결측이 완전히 같은 행에서 발생합니다.** 최대사출압·전환압력·계량시작위치 세 개도 34건이 전부 같은 행입니다. 계량시간과 계량시작점도 9건이 일치합니다.

개념

교안 p36이 말한 **히트맵의 가로 줄무늬**가 바로 이 현상입니다. "**가로 줄무늬 = 그 시점에 여러 센서가 한꺼번에 멈춤.**" 그림을 그리지 않고도 코드로 확인할 수 있습니다. 이런 묶음을 찾으면 "**이 세 항목은 같은 측정 단계에서 나온다**"는 공정 지식을 데이터에서 역으로 읽어 낼 수 있습니다.

설비 적용

실무에서 이 정보는 **처리 방법을 바꿉니다.** 세 컬럼이 항상 함께 빈다면 그것은 **하나의 사건**이지 세 개의 사건이 아닙니다. 각각을 따로 평균으로 채우면 세 값이 서로 모순되는 조합이 만들어질 수 있습니다. 반대로 **셋 중 하나만 남기거나, 세 컬럼을 함께 다루는 편이 안전합니다.**

7-4. 결측이 특정 그룹에 쏠려 있는가

실습 5의 마지막 단계는 **결측이 불량 여부와 관련이 있는지** 확인하는 것입니다. 만약 불량품에서만 결측이 많다면, 그것은 단순 누락이 아니라 **원인이 있는 결측**입니다.

Practice/15_01_example.py — 실습 5

불량여부 그룹별로 결측 상황이 다른지 비교

```
df_5["행결측수"] = na_per_row
```

```
print(
```

```
    df_5.groupby("불량여부")
```

```
    .agg(행수=("행결측수", "count"), 평균결측수=("행결측수", "mean"), 최대결측수=
```

```
    ("행결측수", "max"))
```

```
    .round(2)
```

```
)
```

▶ 행수 평균결측수 최대결측수

▶ 불량여부

▶ 0 217 1.96 8

▶ 1 33 1.48 5

어제 배운 `groupby().agg()`가 여기서 다시 쓰입니다. 결과를 읽어 봅시다. **정상품(0)의 평균 결측이 1.96개, 불량품(1)이 1.48개**입니다. 불량품 쪽이 오히려 적습니다.

개념

결론은 **"결측은 불량 여부와 관련이 없다"**입니다. 만약 불량품에서 결측이 두세 배 많았다면 **"불량이 나는 사이클에서 센서가 제대로 못 읽는다"**는 가설을 세워야 했을 것입니다. 그랬다면 결측을 함부로 채우는 것이 위험합니다. **결측 자체가 불량**의 신호였을 테니까요. 다행히 이 데이터는 그렇지 않으니 마음 놓고 채워도 됩니다.

STEP 8

확인에서 판단으로

비율 기준 · 제거 vs 대체 · 행이나 열이나

오늘 배운 것은 전부 **확인**이었습니다. 어디에 얼마나 결측이 있는지 파악하는 도구들입니다. 수업 마지막 시간대에 강사님은 **판단** 단계로 넘어가는 이야기를 시작하셨습니다.

8-1. 처리의 큰 그림



비유

교안의 비유가 좋습니다. **의사가 진찰하고, 치료 방침을 정하고, 처방을 내리는 흐름과 같습니다.** 어디가 아픈지도 모르고 약부터 먹는 일을 피하는 것이 확인 단계의 목적입니다. 그리고 마지막 **재확인을 빠뜨리면 안 됩니다.** 이 흐름은 일직선이 아니라 **고리**입니다. 결과가 어색하면 다시 확인으로 돌아옵니다.

오늘 배운 15_01은 이 그림에서 **첫 단계인 확인**에 해당합니다. 다음 시간에 배울 15_02가 **판단과 처리**입니다.

8-2. 제거와 대체 — 정반대의 두 접근

구분		
작동 방식	결측이 있는 행·열을 통째로 삭제	빈 칸만 콕 집어 그럴듯한 값으로 채움
데이터 크기	줄어듦	원래 크기 유지
장점	추측이 없어 정직 · 왜곡 없음	데이터를 하나도 안 버림
단점	멀쩡한 값까지 함께 손실	채운 값은 결국 추측
판다스 도구	<code>df.dropna()</code>	<code>df.fillna()</code>

이 데이터에 그냥 제거를 적용하면 어떻게 되는지 7-2에서 이미 확인했습니다. **250행이 76행으로 줄어듭니다. 70% 손실입니다.** 그래서 이 데이터에서는 대체가 거의 필수입니다.

주의

그렇다고 대체가 공짜는 아닙니다. **채운 값은 어디까지나 추측**입니다. 결국 100개를 평균으로 채우면 그 값 하나에 100개가 몰려 **분포가 인위적으로 뽕족해**집니다. 원래 다양하게 퍼져 있어야 할 값들이 한 점에 뭉치는 것입니다. 그래서 교안은 "**결측이 절반을 넘는 컬럼은 차라리 제거가 낫다**"고 말합니다. 6-5의 40% 기준선이 여기서 나옵니다.

8-3. 행을 지울까 열을 지울까

수업의 마지막 대목이자 강사님이 가장 길게 설명하신 부분입니다. 그리고 **오늘 수업에서 유일하게 질문이 오간 지점**이기도 합니다.

결측 분포	판단	이유
일부 행에 집중	행 삭제	측정 실패한 몇몇 행만 빼면 나머지는 거의 살림
특정 센서에 집중	열 삭제	절반 넘게 빈 센서 하나만 빼면 모든 행을 살릴 수 있음

그런데 실무에서는 대체로 **행을 지우는 쪽**을 먼저 고려합니다. 강사님의 설명이 명쾌했습니다.

개념

"아무리 컬럼이 많아진다 해도 컬럼은 수십 개를 넘어가지 않아요. 우리가 봤던 데이터도 컬럼 최대가 한 22 개쯤이었죠. 두 자리 숫자예요 보통."

"컬럼이 그러니까 거기서 하나를 날려 버리면 어마어마한 데이터 손실이 나겠죠. 20분의 1을 날려 버리는 건데."

"그런데 행 같은 경우에는 수가 많아요. 컬럼은 열 개인데 데이터 로우가 4만 개예요. 그중에 몇 개를 지운다고 하면 전체의 퍼센테이지로 보면 0.몇 % 날려 버리는 것밖에 안 되죠."

숫자로 정리하면 이렇습니다. **열은 스무 개 남짓, 행은 수만 개**. 그래서 하나를 지웠을 때의 손실 비중이 완전히 다릅니다.

대상	보통 개수	하나 지울 때 손실	신중함
열 (컬럼)	10 ~ 수십 개	5 ~ 10%	매우 신중해야 함
행 (로우)	수천 ~ 수만 개	0.0몇 %	비교적 자유로움

비유

강사님이 든 비유를 그대로 옮기면 "**소에서 털 하나 뺀다고 그 소가 죽어 버리겠냐**"입니다. 4만 행 중 몇 줄 지우는 것은 그 정도 일이라는 뜻입니다. 반면 **열 하나를 지우는 것은 소의 다리 하나를 떼는 것에 가깝습니다**.

수업 중 오간 질문

이 대목에서 질문이 하나 나왔습니다. "기준 컬럼을 날리지 말고 행을 날리자, 그 말씀이요?" 강사님의 답은 이랬습니다.

주의

"그러니까 의미를 말하는 거예요. 퍼센테이지적으로. 컬럼을 날린다는 건 그만큼 신중해야 된다는 말씀을 드리는 거예요. 실무에서 권장 순서에 열보다는 행을 먼저 따지라고 한 이유는 여기 나온 그 얘기에요." 즉 "열은 절대 지우지 마라"가 아니라 "열을 지울 때는 더 신중하라"는 뜻입니다. 교안 p11의 실무 권장 순서는 열 → 행인데, 이는 결측이 절반 넘게 빈 부실 컬럼을 먼저 정리하고, 그다음에 남은 부실 행을 정리하라는 단계 이야기입니다. 판단의 신중함과 처리의 순서는 별개입니다.

8-4. 이 데이터에 적용해 보면

오늘 확인한 숫자들을 판단으로 옮겨 보겠습니다. 아직 dropna를 배우지 않았지만, 결과를 미리 계산해 두면 다음 시간이 훨씬 수월합니다.

다음 시간에 배울 것을 미리 계산해 보기

```
print("원본      :", df_5.shape)
print("결측 행 전부 삭제 :", df_5.dropna().shape)
print("결측 열 전부 삭제 :", df_5.dropna(axis=1).shape)
print("40%초과 열 제거 후 행 삭제:",
df_5.drop(columns=["계량종료점", "감압시간"]).dropna().shape)
```

```
▶ 원본      : (250, 22)
▶ 결측 행 전부 삭제 : (76, 22)    ← 70% 손실
▶ 결측 열 전부 삭제 : (250, 10)   ← 열이 절반 이하로
▶ 40%초과 열 제거 후 행 삭제: (110, 20)    ← 손실 56%
```

세 결과를 비교하면 판단의 근거가 보입니다. 결측 행을 전부 지우면 76행(70% 손실), 결측 열을 전부 지우면 22열이 10열로 줄어듭니다. 둘 다 너무 큼니다.

반면 결측률 40%를 넘는 두 컬럼만 먼저 빼고 행을 지우면 110행이 남습니다. 손실이 70%에서 56%로 줄었습니다.

컬럼 두 개를 포기한 대가로 행 34개를 더 살린 것입니다. 이것이 "부실 컬럼 먼저, 그다음 부실 행"이라는 단계적 접근의 효과입니다.

개념

그래도 56% 손실은 여전히 큼니다. 결론은 명확합니다. 이 데이터는 제거만으로 해결되지 않습니다. 부실 컬럼 두 개만 제거하고, 나머지 결측은 대체로 채워 250행을 온전히 유지하는 편이 낫습니다. 그 방법이 다음 시간의 fillna입니다.

치트시트

오늘 나온 모든 도구를 한 표에

A-1. 결측 확인 도구

도구	문법	돌려주는 것
<code>.isna()</code>	<code>df.isna()</code>	원본과 같은 크기의 참·거짓 표
<code>.isna().sum()</code>	<code>df.isna().sum()</code>	컬럼별 결측 개수 (Series)
<code>.isna().sum().sum()</code>	<code>df.isna().sum().sum()</code>	전체 결측 개수 (정수)
<code>.isna().mean()</code>	<code>df.isna().mean()</code>	컬럼별 결측 비율 (0~1)
<code>.isna().sum(axis=1)</code>	<code>df.isna().sum(axis=1)</code>	행별 결측 개수 (Series)
<code>.isna().any(axis=1)</code>	<code>df.isna().any(axis=1)</code>	결측이 하나라도 있는 행 (참·거짓)
<code>.info()</code>	<code>df.info()</code>	Non-Null Count로 결측 추정
<code>.describe()</code>	<code>df.describe()</code>	count로 결측 추정 + min·max로 위장 결측 단서

A-2. 복사해 쓰는 골격

데이터를 받으면 이 순서로

```
import pandas as pd
```

```
# 1) 위장 결측을 아는 만큼 처음부터 NaN으로
```

```
df = pd.read_csv("파일.csv", encoding="utf-8", na_values=[ -999, 999])
```

```
# 2) 첫인상
```

```
print(df.shape)
```

```
df.info()
```

```
# 3) 컬럼별 개수와 비율을 한 표로
```

```
report = pd.DataFrame({
```

```
    "결측수": df.isna().sum(),
```

```
    "결측률(%)": (df.isna().mean() * 100).round(1),
```

```
})
```

```
report = report[report["결측수"] > 0].sort_values("결측률(%)", ascending=False)
```

```
print(report)
```

```
# 4) 행 방향 — 다 지우면 몇 줄 남는가
```

```
print("완전한 행:", (~df.isna().any(axis=1)).sum(), "/", len(df))
```

```
# 5) 남은 위장 결측 직접 확인 (컬럼마다 다름)
```

```
print("압력 0 :", (df["사출압력"] == 0.0).sum())
```

A-3. 헛갈리는 이름 정리

이름	정체	쓸까
<code>isna()</code> · <code>isnull()</code>	완전히 같은 기능 — 이름만 둘	isna 하나만
<code>notna()</code> · <code>notnull()</code>	isna의 반대 — 값이 있으면 참	필요할 때만
<code>NaN</code> · <code>None</code>	출신이 다르지만 판다스에선 똑같이 결측	구분 안 해도 됨
<code>count()</code>	결측이 아닌 값의 개수	결측 추정에 유용
<code>size</code>	전체 칸 개수 (결측 포함)	속성이라 괄호 없음

주의

`df.size`는 속성이라 괄호가 없습니다. `df.groupby("A").size()`는 메서드라 괄호가 있습니다. 이름은 같은데 하나는 속성, 하나는 메서드입니다. 읽는 법에서 배운 **3초 판별법**이 여기서 쓰입니다. 점 뒤에 괄호가 붙었는지만 보면 됩니다.

A-4. 오늘의 판단 기준

질문	기준	다음 행동
이 컬럼을 살릴까	결측률 5% 미만	대체로 살리기
	결측률 5 ~ 30%	중요도 보고 결정
	결측률 40% 이상	제거 고민
행을 지울까 열을 지울까	결측이 어디에 몰려 있나	행 집중이면 행, 센서 집중이면 열
제거할까 대체할까	제거 시 남는 행이 절반 아래 인가	절반 아래면 대체로

오류·함정 사전

제출 코드 교정 · 교안 예상 결과 대조

B-1. 제출 코드 검토

오늘 제출하신 두 파일 모두 **문법 오류 없이 정상 실행**되었고, 주석의 관찰도 정확합니다. 특히 좋았던 부분과 손볼 부분을 나눠 정리합니다.

파일	좋은 점	손볼 점
14_03_example.py	표본 수를 함께 출력 · 발견·해석·행동 3단 구성	고장 건수를 비율로도 환산하면 완성
15_01_example.py	진짜/위장 결측 분리 · 변환 전후 비교표 · na_values에 0을 뺀 판단	실습 5에서 원본에 열을 직접 추가

① 15_01_example.py 실습 5 — 원본에 열을 직접 추가

문제 코드

x 원본 데이터프레임에 새 열을 그대로 추가

df_5["행결측수"] = na_per_row

이후 df_5의 모양이 바뀐다

print(df_5.shape)

▶ (250, 23) ← 22열이던 것이 23열로

바로잡기

✓ 사본을 만들어 작업하거나, 임시 표를 따로 만든다

```
tmp = pd.DataFrame({"불량여부": df_5["불량여부"], "행결측수": na_per_row})
print(
    tmp.groupby("불량여부")
    .agg(행수=("행결측수", "count"), 평균결측수=("행결측수", "mean"), 최대결측수=
("행결측수", "max"))
    .round(2)
)
print("원본은 그대로:", df_5.shape)
```

▶ 행수 평균결측수 최대결측수

▶ 불량여부

▶ 0 217 1.96 8

▶ 1 33 1.48 5

▶ 원본은 그대로: (250, 22)

결과는 똑같지만 **원본이 오염되지 않습니다**. 이 코드에서는 마지막 단계라 문제가 없었지만, 만약 뒤에

`df_5.isna().sum()`을 한 번 더 실행한다면 **새로 추가한 행결측수 열까지 포함되어** 결과가 달라집니다. 지난 시간에 배운 `.copy()` 습관과 같은 맥락입니다.

② 14_03_example.py 실습 4 — 건수를 비율로

최종 해석의 발견 문장이 **"고장 건수는 A라인(16건)이 가장 많음"**입니다. 맞는 관찰이지만 라인마다 측정 건수가 다르므로 비율로 확인하는 편이 안전합니다. 3-3에서 계산한 결과 **A라인 29.6% · C라인 19.4% · B라인 17.1%**로 순위가 유지되어 결론은 그대로입니다. 다만 **확인하고 넘어간 것과 확인하지 않고 넘어간 것은 다릅니다**.

B-2. 교안 예상 결과 대조

교안 15_01은 **SECOM 반도체 공정 데이터(센서01~20)**를 기준으로 쓰였는데, 실제 실습에서는 **사출성형 공정 데이터**를 썼습니다. 그래서 컬럼 이름이 다릅니다. 그런데 숫자는 대부분 그대로 맞습니다.

교안	교안이 말한 것	실제 실행 결과
15_01 p21	실습 1 — "NaN 4개 (온도2·압력1·진동…)"	실제 5개 (사출기1·배럴온도2·사출압력1·스크루속도1)
15_01 p31	전체 결측 475개	일치
15_01 p32	최상위 43.6% · 그다음 27.2% · 24.0%	일치 (컬럼 이름만 다름)
15_01 p33	결측 없는 행 76개 · 있는 행 174개	일치
15_01 p40	실습 3 — 변환 후 NaN 5개 → 8개	일치

교안	교안이 말한 것	실제 실행 결과
15_02 p4	dropna 후 250행 → 76행	일치
15_02 p5	dropna(axis=1) 후 22열 → 10열	일치

개념

교안 숫자가 이렇게 잘 맞는 것은 오랜만입니다. 두 데이터가 이름만 다를 뿐 **결측 구조를 똑같이 만들어 둔 교육용 데이터**이기 때문입니다. 유일하게 어긋난 p21의 "NaN 4개"도 예시 컬럼 이름(온도·압력·진동)이 실제 데이터와 달라 생긴 차이로 보입니다. 그래도 **직접 실행해 확인하는 습관은 그대로 유지하십시오**. 맞는 것을 확인하는 데도 같은 시간이 듭니다.

B-3. 오늘 만나기 쉬운 오류

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>df[df["온도"] == np.nan]</code>	언제나 0건 — 결측을 못 찾음	<code>df[df["온도"].isna()]</code>
<code>na_values=[0, -999]</code>	정상값 0까지 결측으로 바뀜	<code>na_values=[-999]</code> 후 0은 따로 판단
<code>df.isna().sum(1)</code>	동작은 하지만 의도가 안 보임	<code>df.isna().sum(axis=1)</code>
<code>df.size()</code>	TypeError — 속성에 괄호	<code>df.size</code>
<code>report.결측률(%)</code>	SyntaxError — 이름에 특수문자	<code>report["결측률(%)"]</code>
<code>df.dropna()</code>	저장 안 하면 원본은 그대로	<code>clean = df.dropna()</code>

자주 하는 실수

마지막 줄이 결측 처리에서 **가장 흔한 실수**입니다. `df.dropna()`는 새 데이터프레임을 만들어 돌려줄 뿐 **원본을 바꾸지 않습니다**. 결과를 변수에 담지 않으면 화면에는 처리된 듯 보이지만 원본은 그대로입니다. 다음 시간에 배울 `fillna()`도 마찬가지입니다. **"처리했는데 왜 안 바뀌지"**의 심증팔구가 이것입니다.

실습 하이라이트와 다음 시간

핵심 결론 정리 · 15_02 본론 예고

C-1. 오늘 얻은 여섯 가지 결론

#	결론	근거
1	전체를 볼 때와 그룹을 나눠 볼 때 결론이 뒤집힐 수 있습니다.	지표07-08 상관: 전체 -0.969 vs 합격 +0.385
2	표본 12건의 상관계수는 믿을 수 없습니다.	실제 0.385인 집단에서 12건 뽑으면 46.7%가 $ r \geq 0.7$
3	결측은 오류를 내지 않고 조용히 계산을 바꿉니다.	배열온도 16행 중 14개로 평균 계산 — 경고 없음
4	등호로는 결측을 찾을 수 없습니다.	<code>np.nan == np.nan</code> 이 False — 반드시 <code>isna()</code>
5	진짜 결측과 위장 결측이 반반이었습니다.	로그 데이터: NaN 5개 + 위장 5개 = 총 10개
6	세 컬럼이 항상 함께 빕니다.	최대사출압·전환압력·계량시작위치 34건이 전부 같은 행

설비 적용

6번이 오늘 가장 실무적인 발견입니다. 세 컬럼이 **항상 같은 행에서 함께 빕니다**는 것은 우연이 아니라 **하나의 사건**입니다. 계량 단계에서 측정이 끊기면 그 단계의 세 항목이 통째로 기록되지 않는 것입니다. 처리할 때도 셋을 **따로따로가 아니라 한 덩어리**로 다뤄야 합니다. 각각을 자기 컬럼 평균으로 채우면 물리적으로 앞뒤가 안 맞는 조합이 만들어질 수 있습니다.

C-2. 오늘 푼 실습 정리

교안	실습	핵심 결과
14_03	1. 두 열의 상관계수	지표07-08 -0.969 · 부호는 방향, 절댓값은 강도
14_03	2. 상관 행렬로 여러 열 비교	10×10 표 · 한 열만 뽑아 정렬하는 방법
14_03	3. 그룹별 상관 비교	전체 -0.969인데 합격만 보면 +0.385로 뒤집힘
14_03	4. 통합 리포트 종합	발견·해석·행동 3단 · A라인 고장 16건, C라인 편차 10.38
15_01	1. 눈으로 결측 찾기	진짜 5개 + 위장 5개 — <code>isna</code> 만으로는 절반만 잡힘

교안	실습	핵심 결과
15_01	2. 공정 데이터 첫 탐색	250×22 · 결측 있는 컬럼 12개 · 뒤쪽일수록 많음
15_01	3. 위장 결측 사냥	na_values로 NaN 5개 → 8개 · 0은 일부러 남김
15_01	4. 결측 비율 계산	최대 43.6% 두 컬럼 · 40% 기준선 넘음
15_01	5. 행 단위 결측 패턴	완전한 행 76개(30.4%) · 불량 여부와 무관

교안 15_01의 **실습 6(결측 시각화)**과 **실습 7(요약표 저장)**은 하지 않았습니다. 시각화는 아직 배우지 않은 도구가 필요하고, 요약표 저장은 4번 실습의 표를 파일로 내보내는 것이라 문법이 새로 나오지 않습니다.

C-3. 다음 시간 예고 — 15_02 결측치 제거와 대체

오늘 개념만 짚고 넘어간 15_02의 본문입니다. 배울 도구가 많습니다.

도구	하는 일	미리 계산한 결과
<code>dropna()</code>	결측 있는 행 삭제	250행 → 76행 (70% 손실)
<code>dropna(axis=1)</code>	결측 있는 열 삭제	22열 → 10열
<code>dropna(how="all")</code>	모든 값이 빈 행만 삭제	250행 그대로 (완전 빈 행 없음)
<code>dropna(thresh=20)</code>	값이 20개 이상인 행만 남김	250행 → 162행 (35% 손실)
<code>dropna(subset=[...])</code>	특정 컬럼이 빈 행만 삭제	정답 컬럼 보호에 사용
<code>fillna(값)</code>	빈 칸을 지정한 값으로 채움	상수 · 평균 · 중앙값 · 최빈값
<code>ffill()</code> <code>bfill()</code>	앞뒤 값으로 채움	시계열 데이터에 적합

개념

`thresh`는 특히 헛갈립니다. **결측 개수가 아니라 살아 있는 값의 최소 개수**를 정하는 것입니다. `thresh=20`은 "값이 20개 이상 있는 행만 남겨라"는 뜻이고, 22개 컬럼이니 **결측 2개 이하인 행만 살아남습니다**. 방향을 반대로 이해하기 쉬우니 조심하십시오. 이 데이터에서는 162행이 남아 손실이 35%로 줄어듭니다. 그냥 `dropna()`의 70% 보다 훨씬 낫습니다.

대체 쪽에서는 **어떤 값으로 채울지 고르는 안목**이 핵심입니다. 어제 배운 평균과 중앙값의 차이가 여기서 바로 쓰입니다.

데이터 종류	대체 방법	이유
숫자 · 고른 분포	평균	이상치가 없으면 가장 자연스러운 대표값
숫자 · 치우친 분포	중앙값	이상치에 흔들리지 않는 안전한 선택

데이터 종류	대체 방법	이유
글자 · 분류	최빈값	평균을 낼 수 없으니 가장 자주 나오는 값
시계열	앞뒤 값	설비 상태는 짧은 시간에 급변하지 않음
그룹 특성이 뚜렷	그룹별 평균	한 덩어리 평균보다 그룹 평균이 정확

연결

어제 배운 "평균은 극단값에 끌려가고 중앙값은 안 흔들린다"가 곧바로 실전 판단이 됩니다. 대체하기 전에 그 컬럼의 평균과 중앙값을 둘 다 구해 비교해 보고, 비슷하면 평균, 많이 다르면 중앙값을 고릅니다. 어제 배운 것이 오늘 도구가 되고 내일 판단 기준이 됩니다.

C-4. 복습 순서

순서	무엇을	시간
1	Practice/15_01_example.py를 처음부터 다시 실행하며 출력 확인	15분
2	부록 A-2의 골격을 15_02_사출성형_공정.csv에 그대로 적용해 보기	15분
3	7-3의 동시 결측 그룹 찾기 코드를 직접 쳐 보고 결과 해석하기	15분
4	<code>np.nan == np.nan</code> 을 실제로 실행해 False를 눈으로 확인하기	3분
5	8-4의 네 가지 삭제 방식을 직접 실행해 shape 비교하기	10분

C-5. 스스로 확인하는 질문 여섯 개

아래 여섯 질문에 자료를 보지 않고 답할 수 있으면 오늘 분량은 충분히 소화한 것입니다.

#	질문	확인 지점
1	<code>isna()</code> 가 돌려주는 것은 무엇입니까	원본과 같은 크기의 참·거짓 표 — 6-1
2	<code>sum()</code> 과 <code>mean()</code> 은 무엇이 다른니까	개수와 비율 — 6-4
3	<code>axis=1</code> 은 어느 방향입니까	가로 — 행마다 계산 — 7-1
4	<code>== np.nan</code> 이 안 되는 이유는 무엇입니까	NaN은 자기 자신과도 다름 — 4-5
5	<code>na_values</code> 에 0을 넣으면 안 되는 경우는	0이 정상값일 수 있을 때 — 5-4
6	열보다 행을 먼저 지우는 이유는 무엇입니까	열은 수십 개, 행은 수만 개 — 8-3

오늘의 성취

빈 칸을 발견하는 눈

연결

앞 표의 6번이 오늘 처음 등장한 사고입니다. 나머지 다섯은 도구의 사용법이지만, **6번은 도구를 쓰기 전에 방향을 정하는 판단**입니다. 이 질문에 답할 수 있으면 다음 시간에 배울 `dropna`의 여러 옵션을 훨씬 쉽게 이해할 수 있습니다.

오늘 배운 도구는 사실 몇 개 되지 않습니다. `isna`, `sum`, `mean`, `axis`, `na_values`. 이 다섯이 전부입니다. 그런데 이 다섯으로 할 수 있는 일이 상당히 많았습니다.

개념

계산이 조용히 틀리는 상황을 알아채는 것. 이것이 오늘의 핵심입니다.

결측이 있어도 에러는 나지 않습니다. 평균이 나오고 상관계수가 나옵니다. 다만 그 숫자가 **빠진 값을 뻔 채로 계산된 숫자**일 뿐입니다. 배럴온도 평균 257.9도가 그 예입니다. 999 하나 때문에 실제보다 57도나 높게 나왔는데, 화면에는 아무 경고도 없었습니다.

그래서 데이터를 받으면 `info()`와 `isna().sum()`을 반드시 먼저 **실행합니다. 이것이 오늘 얻은 습관**입니다.

한 걸음 더 나아가면, 오늘 배운 것은 **데이터를 의심하는 법**입니다. 지금까지 22일 동안 우리는 주어진 숫자를 어떻게 요약할지 배웠습니다. 오늘부터는 **그 숫자가 믿을 만한 것인지 먼저 묻습니다.** 순서가 뒤바뀐 것처럼 보이지만, 실무에서는 이 질문이 언제나 먼저입니다.

어제까지	오늘 더한 것	다음에 만날 것
숫자를 요약하기	그 숫자를 믿을 수 있는지 확인하기	빈 칸을 없애고 다시 요약하기
<code>groupby</code> · <code>agg</code> · <code>corr</code>	<code>isna</code> · <code>na_values</code> · <code>axis=1</code>	<code>dropna</code> · <code>fillna</code>
깨끗한 데이터 전제	현실의 지저분한 데이터 직시	처리하고 검증까지

마지막으로 오늘 확인한 숫자 하나만 다시 짚겠습니다. **250행 중 결측이 전혀 없는 행은 76행, 30.4%뿐입니다.** 나머지 70%에는 어딘가 빈 칸이 있습니다. 이 데이터를 그냥 분석에 넣었다면 어떻게 됐을까요. 아마 에러 없이 결과가 나왔을 것이고, 그 결과가 틀렸다는 사실도 몰랐을 것입니다. **빈 칸을 찾아낸 오늘의 작업이 그 사고를 막았습니다.**

— 22일차 정리 끝. 다음은 15_02 결측치 제거와 대체입니다.